

Reactive — When Proxy Is Asked

What happens the moment a person turns to Proxy and asks for something. Part A is the product — every kind of help it gives, for any request, at any company. Parts B–D are the build — the exact path an ask travels, the five pieces of intelligence that make it feel like a sharp colleague rather than a command line, and the latency work that makes it fast enough to talk to.

Deep Dive 01 built the knowledge; Deep Dive 02 built the live understanding of the room. This is the first way that understanding is *provoked*: someone addresses Proxy directly — "Proxy, run the load test," "help me write this migration," "walk me through how billing retries," "is that number right?" Reactive is Proxy answering. It is the simplest thing to explain and the thing people touch first, so it has to be right.

THE ONE IDEA TO HOLD

Being asked **is permission**. So a human request doesn't get its own special engine — it drops into the *same* loop Proxy uses to think for itself (Deep Dive 04), just as the **highest-priority** item, with the "should I even speak?" gate switched off because the person already asked. And "asked for what" is deliberately wide: Proxy can **do it**, **help you do it**, **explain it**, or **tell you if it's right** — four equal kinds of help, not one. Execution is only one of the four.

PART A — THE PRODUCT

Every kind of help, for any ask, at any company

What "reactive" delivers as a product — written before any implementation detail.

When a person addresses Proxy, they can be asking for any of four fundamentally different things. Most meeting tools do zero of them; the few that do anything do only the first. Proxy treats all four as first-class, because a good colleague does all four without being told which one you meant.

The four modes of help

- **Do it (execute).** "Proxy, run the checkout load test at 2× traffic." Proxy does the actual work against the real systems and hands back a result with a receipt — the test ran, here is the exit code, here is the output. This is the mode people picture first, and it is exactly one quarter of what reactive is.

- **Help me do it (assist / coach).** "Help me write the rollback script." Here Proxy does *not* take over — it works alongside your hands, step by step, at a conversational pace. Two shapes of the same thing: either Proxy *coaches you through your own hands* ("open `deploy.yml` — now change line 40 to...") or *you narrate and Proxy executes on its own canvas* ("add a health check before the swap" → Proxy types it into its tile for you to watch and approve). This is the "guide-me" pattern: collaboration at a step-paced cadence, which is just reactive help delivered one move at a time instead of all at once.
- **Explain it / walk me through it (teach).** "Walk me through how billing handles a timeout." Proxy teaches — at *the asker's* depth. A new engineer gets the plain-English tour; a staff engineer gets the exact retry policy, the file and line, and the edge case. It reads the estate (Deep Dive 01) and narrates the real thing, not a generic tutorial.
- **Is this right? / What do you think? (advise).** "We're about to promise 99.9% uptime — is that safe?" Proxy advises — it checks the claim against what the systems actually do and gives you a grounded opinion with the evidence attached, not a vibe. "You run at 99.7% today with no EU redundancy; 99.9% needs the failover work first."

WHY FOUR AND NOT ONE

The mode is never chosen from a menu, by Proxy or by the asker. The person just talks; Proxy reads what shape of help the sentence wants. "Do X" is execute. "Help me X" is assist. "How does X work" is teach. "Is X right" is advise. The same underlying machinery serves all four — the only difference is what comes back and on which surface.

Chat is a first-class compute lane

You can **type** to Proxy in the meeting chat exactly as you'd speak to it, and it is not a lesser channel. Type a question → Proxy computes it against the whole estate → it answers with a receipt, right there in chat. This matters for two reasons. First, a chat ask is *private and non-disruptive* — you can quietly ask "is that 2% number real?" without interrupting the speaker. Second, and this is the design point: **a typed ask and a spoken ask are the identical operation** — same loop, same grounding, same verification — differing in nothing except the surface the answer returns on (text in the chat panel vs. a spoken line or a card on the tile). There is no separate "chatbot" inside Proxy; chat is just the meeting's keyboard wired into the same brain.

It covers any action, at any company

Reactive is not a fixed list of commands. Whatever Proxy's hands can do — run code, fire a live query, build a spreadsheet or a mock-up, draft a document, stage a pull request, look up a live number, register a watcher — a person can ask for by name. New capabilities are added to the hands (Deep Dive 05), and reactive can invoke them the same day with no change to this path. And because nothing here branches on what the meeting is about, the identical mechanism serves a two-person startup standup and a board review at a ten-thousand-person enterprise: only the connected systems and the resolved referents differ. There is no vertical-specific or size-specific logic anywhere in the reactive path.

What it refuses to do

An impossible or out-of-scope ask does not get improvised. If someone asks for something the systems can't answer or Proxy isn't permitted to reach, it declines honestly — "I can't see the production billing DB; I only have read access to the code" — and files a tracked follow-up so the gap is visible rather than silently guessed around. A confident wrong answer is the one failure reactive is built never to produce.

PART B — HOW IT WORKS

The path an ask travels

One picture, then the flow in words.



Figure 1 — A human ask enters the same loop Proxy uses to think unprompted (Deep Dive 04), but as the top-priority item. It flows through the identical develop → validate → deliver machinery. The one difference from the proactive path: the "should I speak?" gate is bypassed, because the person already asked.

Read the flow left to right. Someone **asks**, by voice or by typing in chat. Because the Meeting Page (Deep Dive 02) is already live, the words in the ask are *already grounded* — "that migration" was resolved to a specific pull request the moment it was first mentioned, so Proxy doesn't have to guess what "that" means; it looks it up on the page. The ask becomes a **maximum-priority candidate** in the same ledger that holds Proxy's own unprompted ideas — a single shared to-do list where a human request simply outranks everything. It flows through **develop** (the hands do whatever the ask needs — run, look up, build, draft), then **validate** (the result is checked by the verifier for its class), then **deliver** (the answer returns on the right surface with its receipt).

The one thing that is *different* from Proxy thinking for itself is drawn in orange: the **surfacing gate is bypassed**. When Proxy speaks up unprompted, a gate decides whether the thought is worth the room's attention. When a human asks, that question is already answered — *you asked, so of course you get an answer*. Everything else — the grounding, the work, the verification — is byte-for-byte the same machinery. This is why reactive and proactive are honestly "one engine at two provocations," not two systems.

WHY REUSE THE PROACTIVE LOOP INSTEAD OF A DEDICATED COMMAND HANDLER?

Because a colleague's answer to "is that right?" and a colleague *noticing* "that's not right" require the exact same work: resolve the referent, check it against the live system, attach the proof. Building two engines would mean building that work twice and letting them drift apart. One ledger, one develop-validate-deliver path, one set of verifiers — the ask just enters at the front of the line.

The five pieces of intelligence — what each is, how it works, how we optimize it

The bare loop above would answer correctly but bluntly. These five refinements are what make it feel like a sharp colleague. For each: the overview first, then the mechanism and its tuning.

A literal "resolve → do → verify → answer" is correct but clumsy — it would ask needless questions, sit silent while it thinks, and run side-effecting work with no chance to catch a misunderstanding. Five refinements fix that. Every one of them rides pieces that already exist in Deep Dives 01, 02, and 05; none is a new subsystem.

Refinement 1 — Clarify-or-declare: ask a question only when it would change the outcome

Overview. The dumbest thing an assistant can do is ask you to confirm what it already knows; the second dumbest is to guess silently and act on the wrong thing. Clarify-or-declare threads between them. Before doing the work, Proxy runs a cheap check on how sure it is about the *referent* (what "that" points at), the *scope* (how much of it), and the *intent* (which of the four modes). It asks a question **only** when confidence is below a floor *and* the answer would actually change what it does. Otherwise it acts — but *declares its assumption aloud* so you can veto it mid-flight.

How it works. A small, fast model (an "ambiguity classifier") scores the ask against the Meeting Page's resolved referents. Concretely: "re-run *it* at 2× load" — the classifier checks whether "it" resolves to exactly one prior action on the page. If yes and confidence is high, Proxy just runs it. If "it" could mean either of two recent runs, and picking wrong wastes real work, that clears the bar for *one* tight question — with the best guess already embedded so a "yes" is one word: "the checkout load test, right? running it at 2×." If the ambiguity *wouldn't* change the action (e.g., an unclear detail that both readings share), it doesn't ask at all — it acts and declares: "running the checkout test at 2× — assuming production config; say stop if you meant staging."

SYSTEM PROMPT — CLARIFY-OR-DECLARE CHECK (ILLUSTRATIVE)

You gate a human ask before Proxy acts. You do NOT answer it.
Inputs: the ask, the Meeting Page's resolved referents, the four modes.

For each of {referent, scope, intent}, output a confidence 0–1 and the candidate values you're choosing between.

Decide ONE of:

- ASK — only if min-confidence < floor AND the alternatives would lead to DIFFERENT actions. Emit ONE question with the best guess embedded.
- DECLARE— confidence is fine, OR the ambiguity doesn't change the action. Emit the single assumption to state aloud while acting.
- GO — unambiguous; act silently.

Never ask to confirm something already resolved on the page. Never ask a question whose answers all produce the same action. One question maximum.

How we optimize it. Three ways. (1) *It runs on the cheapest model tier* (Haiku-class) because it's a scoring task, not a reasoning task — cents per meeting. (2) *It reads the page, not the transcript*, so its input is tiny and constant-size no matter how long the meeting has run. (3) *The floor is learned per room* — the same logging that trains everything else records when a declared assumption got vetoed (floor too low, should have asked) or a question got an eye-roll "obviously yes" (floor too high, should have acted), and nudges the threshold. Over weeks it asks fewer, better questions at this specific company.

Refinement 2 — Speculative pre-work: start before the sentence ends

Overview. Much of the latency in answering is grounding — resolving what you meant and fetching the current facts. Speculative pre-work moves that *before* the ask is even finished. The instant someone says the wake word "Proxy—", the page already shows what's being discussed, so Proxy pre-resolves the likely referent, warms the owning expert's cache, and fires the *read-only* first step — so by the time the sentence lands, half the work is done.

How it works. The wake word is the trigger. On "Proxy—", three things fire in parallel, all side-effect-free: (1) the most-recently-referenced estate node on the page is pre-resolved as the probable target; (2) that node's expert (Deep Dive 01) is summoned to the bench and its prompt-cache warmed, so it's ready to answer instantly; (3) if the likely ask is a lookup, the read-only first step — a live query for the current value — is fired speculatively. If the actual ask matches the guess, the answer is already in hand. If it doesn't, the speculative work is simply discarded — and because it was strictly read-only, discarding it costs nothing but a few cents of compute.

WHY THIS IS SAFE, AND BOUNDED

Speculation only ever *reads* — it never writes, never stages a draft, never touches a real system's state. The worst case of a wrong guess is a wasted read. And it is capped per meeting (a small budget of speculative fires) so a chatty room can't turn it into a cost leak. Side-effect-free and bounded: those two properties are what make "start before you're sure" a safe optimization rather than a reckless one.

How we optimize it. The heavy lifting is done by the prefetcher (Deep Dive 04) — a piece of *plumbing, not a model*: when the page marks something worth watching, deterministic live reads fire into short-lived, timestamped evidence bundles. Speculative pre-work just points that same prefetcher at the wake word. So the optimization is essentially free: we reuse a component that already exists for the proactive path, and the payoff is that the most common reactive ask — "Proxy, what's the current X?" — is often answered in the time it takes to finish the sentence.

Refinement 3 — Plan-flash: show the plan before multi-step or side-effecting work

Overview. For anything with more than one step, or anything that changes a real system, Proxy flashes a tiny 2–4 step plan *before* it runs — on the tile or the panel. It proceeds by default unless someone objects, so it's not a nagging confirmation dialog; it's a windshield. For side-effecting asks, that flashed plan *is* the confirmation surface — the place a misunderstanding dies cheaply, before any work runs, instead of expensively, after.

How it works. When the hands (Deep Dive 05) determine an ask is multi-step or mutating, the plan they synthesize — which they build anyway, as an internal step — is surfaced as a short human-readable list: "1. pull current checkout config · 2. clone to sandbox · 3. run load test at 2× · 4. report p99." It appears for a beat. Silence means go. An objection ("no, staging, not prod") edits the plan before step one runs. For a read-only lookup, no plan-flash is needed — there's nothing to misunderstand and nothing to undo. For anything that stages a draft or runs real work, the flash is mandatory.

How we optimize it. The plan is *not* extra work — the hands compose a structured plan for every WORK-class job regardless (it's how they decide which tools and sandbox to use). Plan-flash just *renders* that existing artifact to a human. So the cost is one cheap formatting pass, not a second round of thinking. And because the plan is generated with constrained decoding into a fixed schema (Deep Dive 05), it's always well-formed — there's no risk of flashing a garbled plan. The

optimization is: reuse the plan the system already had to make, and spend a few tokens turning it into four readable lines.

Refinement 4 — Stakes-scaled verification: check harder when the answer matters more

Overview. Not every answer deserves the same scrutiny. "What's checkout's p99 right now?" asked idly wants one quick live read. The *same* question, asked because the room is about to commit to a hard-to-reverse decision that depends on the answer, deserves the test actually run and a second independent source. Stakes-scaled verification reads how consequential the answer is — from the page — and scales its checking depth automatically. You never ask for "more rigor"; the situation supplies it.

How it works. The Meeting Page annotates every forming decision with its *reversibility* and its *blast-set* — which estate units it touches (Deep Dive 02's derived signals). When an ask feeds a decision that's moving toward final and is hard to undo, the verification tier steps up: a casual lookup gets the standard single check; a decision-justifying answer gets the test executed *and* a corroborating second source before it's spoken. The scaling is a lookup against fields already on the page — no separate judgment call, no model needed to decide "is this important."

WHY REVERSIBILITY, NOT TOPIC, IS THE DIAL

Scaling by *how hard the decision is to take back* — rather than by keyword or meeting type — is what keeps this vertical-agnostic and honest. "Ship Friday" in a startup standup and "approve the merger data" in a board review are treated with the same rule: how much does it cost if this answer is wrong and someone already acted on it? High cost buys deeper verification. That's a property of the situation, readable from the page, not a hardcoded list of "important topics."

How we optimize it. The default is cheap and only the rare high-stakes ask pays for the deep check — so the *average* cost stays low while the *worst* case (a wrong number feeding an irreversible decision) is the one we spend on. This is the right place to spend: the expensive verification runs a handful of times per meeting, exactly when a mistake would be most costly, and never on the idle lookups that make up most asks.

Refinement 5 — Follow-up continuity: answers that compose

Overview. Proxy's own actions live on the Meeting Page, so a follow-up composes against what came before. "Run it again at 2× load" knows what "it" was and what the last run showed; "how does that compare to last week?" diffs against the prior result. You have a running conversation with

your work, not a series of amnesiac one-shots. And a reactive ask can register a temporary, meeting-scoped **watcher** — "keep an eye on the deploy and tell us if errors spike" — which is the natural bridge from asked behavior into ongoing unprompted behavior, on demand.

How it works. Every action Proxy takes is logged as a result on the page and in the ledger, with its inputs and outputs. A follow-up ask resolves its referents ("it", "that", "the same thing") against those logged actions exactly the way any pronoun resolves against any prior utterance — the machinery is identical, because Proxy's own actions are just more entries on the page. "Run it again at 2× load" therefore resolves "it" to the last load test, inherits its config, and changes only the load parameter. A watcher is a special result: a standing, meeting-scoped instruction that fires a check on an event ("errors spike") and surfaces if it trips — it lives until the meeting ends, then is cleaned up.

How we optimize it. Composition is free because it reuses the page's referent resolution — there's no separate "conversation memory" to build or pay for; Proxy's actions are on the same page as everyone's words, resolved by the same resolver. The watcher is the one piece with an ongoing cost, and it's optimized by being an *event-driven* check (it wakes on the event, not on a polling loop) and *meeting-scoped* (it can't outlive the meeting and leak cost). One cheap standing check, torn down at close.

Latency discipline — the thing that makes it feel like talking to a person

Reactive lives or dies on speed. Correctness is table stakes; a correct answer that arrives four seconds into an awkward silence has already failed as a conversation. This is where most of the reactive optimization work lives.

The target is that an answer *begins* within a couple of seconds — because that's the window in which a spoken reply still feels like a reply and not a delay. Hitting that target is not one trick; it's a discipline built from four rules.

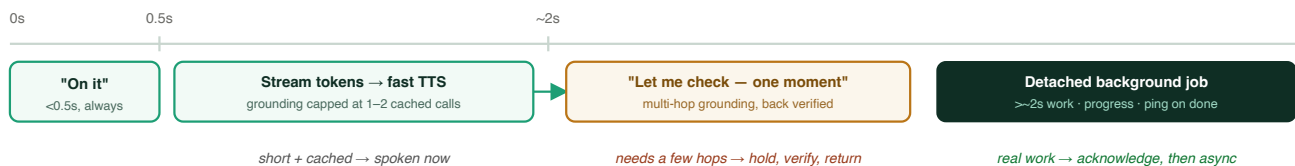


Figure 2 — The latency ladder. The eager "on it" always lands in under half a second; where the answer goes from there depends only on how much grounding it needs. Acknowledge-then-async is structural: the voice channel is never blocked waiting on work.

- **Eager acknowledgment in under half a second.** The instant Proxy is addressed, it says "on it" — a visible or audible acknowledgment before any work starts. This is cheap (no computation) and it's the single most important latency trick, because it converts dead air into a felt response. The human now knows they were heard, which buys the seconds the real answer needs.
- **Tokens stream straight into fast text-to-speech.** For answers Proxy can give from cache or a single quick read, it doesn't wait to finish thinking before it starts speaking — the answer's tokens flow directly into a fast TTS engine as they're generated, so the reply *starts* immediately and completes as it goes. Grounding on this synchronous, spoken path is **hard-capped at one or two cached tool calls** — anything needing more can't run here, by rule, because it would blow the budget.
- **Multi-hop grounding gets an honest "let me check."** If an answer genuinely needs several dependent lookups — resolve A to get B to check C — Proxy doesn't try to force that onto the spoken path and arrive fast-and-wrong. It says "let me check — one moment," does the hops, and

comes back *verified*. Honest and slightly slower beats instant and wrong, every time, on a channel where a wrong number can move a decision.

- **Anything over ~2s becomes a detached background job.** Real work — a load test, a model, a monorepo build — is acknowledged immediately, then detached: it runs as a background job with visible progress, and the requester is pinged when it's done with the artifact and receipt (in the panel now, or via Slack/Teams after the meeting if it runs long). "That'll take about twenty minutes — I'll have it in your Slack before your next meeting" is a first-class answer.

Acknowledge-then-async is structural: the voice channel is *never* blocked by execution, so Proxy stays responsive to the next ask even while a big job runs.

EVERY RETURNED FACT CARRIES ITS PROOF

Speed never comes at the cost of the proof. Whatever path an answer took, before it's spoken or shown it passes the verifier for its *class*: code is verified by running it and checking the exit code; a live fact by re-retrieving it from the source; a computed model by recalculating it. The receipt — what ran, from where, when — rides along with the answer. Fast and proven are not in tension here; the eager "on it" buys the time, and the verifier spends it well.

Why this stays cheap at any scale

- **Most asks are the cheap path.** The great majority of reactive asks are a lookup or a known fact — answered from a warm cache or one live read, on the cheapest model tier, for cents. The expensive machinery (deep verification, real workshop jobs) engages only when the ask genuinely demands it, a handful of times a meeting.
- **Constant-size inputs.** Every piece of the reactive path — the ambiguity classifier, the referent resolution, the verifier — reads the small Meeting Page, never the growing transcript. So a three-hour meeting costs the same per ask as a ten-minute one.
- **The plumbing is the spend, and it scales down.** As with the whole product, the model intelligence in reactive is cents; the real cost is transport and speech-to-text, which come down with vendor selection and self-hosting without touching answer quality.
- **It learns to be cheaper.** The per-room learning that tunes the clarify floor, and the fine-tuned small models trained on the org's own accepted/rejected answers, make reactive both cheaper and more accurate the longer a company uses it.

Done when

Reactive is complete when: an ask with a resolvable referent executes against the *correct* entity on the first try at or above the accuracy floor; the first audible or visible response lands within the latency budget (the eager "on it" in under half a second, the answer beginning within a couple of seconds); every returned fact has passed its class verifier — test exit code for code, live retrieval for facts, recalculation for models; an impossible or out-of-scope ask produces an honest decline plus a tracked follow-up, never an improvisation; a chat ask behaves identically to a voice ask apart from the return surface; an ambiguous ask that would change the action produces exactly one clarifying question and all others act-and-declare; speculative pre-work is provably side-effect-free and bounded; every multi-step or side-effecting ask flashes its plan before running; verification depth provably scales with the decision stakes on the page; a follow-up ask composes against the prior action's logged result; and all four modes — do it, help me do it, explain it, advise — are served by the identical loop, with only the returned help differing.